SHREE VENKATESHWARA

HI-TECH ENGINEERING COLLEGE

(AUTONOMOUS)

GOBI-638 455

NAME OF THE LAB_____

University Register No_____

Name _____ Roll No_____

Branch_____ Batch_____

Certified that this is bonafide record of work done by the above student during the year 2025-2026

LAB IN-CHARGE **HEAD OF THE DEPARTMENT**

Submitted for the practical Examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

23ADL41 **MACHINE LEARNING LABORATORY** L T P C
0 0 4 2

COURSE OBJECTIVES:

- To understand the data sets and apply suitable algorithms for selecting the appropriate features for analysis.
- To learn to implement supervised machine learning algorithms on standard datasets and evaluate the performance.
- To experiment the unsupervised machine learning algorithms on standard datasets and evaluate the performance.
- To build the graph based learning models for standard data sets.
- To compare the performance of different ML algorithms and select the suitable one based on the application.

LIST OF EXPERIMENTS:

1. For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm to output a description of the set of all hypotheses consistent with the training examples.
2. Write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.
3. Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.
4. Write a program to implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file and compute the accuracy with a few test data sets.
5. Implement naïve Bayesian Classifier model to classify a set of documents and measure the accuracy, precision, and recall.
6. Write a program to construct a Bayesian network to diagnose CORONA infection using standard WHO Data Set.
7. Apply EM algorithm to cluster a set of data stored in a .CSV file. Use the same data set for clustering using the k-Means algorithm. Compare the results of these two algorithms.
8. Write a program to implement k-Nearest Neighbour algorithm to classify the iris data set. Print both correct and wrong predictions.
9. Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select an appropriate data set for your experiment and draw graphs.

The programs can be implemented in either Python or R.

TOTAL:60 PERIODS

COURSE OUTCOMES:

At the end of this course, the students will be able to:

CO1:Apply suitable algorithms for selecting the appropriate features for analysis.
CO2:Implement supervised machine learning algorithms on standard datasets and evaluate the performance.
CO3:Apply unsupervised machine learning algorithms on standard datasets and evaluate the performance.
CO4:Build the graph based learning models for standard data sets.
CO5:Assess and compare the performance of different ML algorithms and select the suitable one based on the application.

ANACONDA

Anaconda is a free, open-source distribution of **Python** (and R) used for:

- Data science
- Machine learning
- Scientific computing

It simplifies managing libraries, dependencies, and environments.

Main Components of Anaconda

Component	Description
Conda	A package and environment manager
Anaconda Navigator	A graphical user interface (GUI) to manage environments and launch tools
Jupyter Notebook	A tool for writing and running Python code in your browser
Spyder IDE	An integrated development environment for Python (like PyCharm or VS Code)

Key Features

- Comes with **hundreds of pre-installed libraries** (e.g., NumPy, Pandas, Matplotlib)
- Allows you to create **separate environments** for different projects
- Helps avoid version conflicts between packages
- Works offline after installation

Basics of SPYDER IDE (Scientific Python Development Environment)

What is Spyder?

Spyder is a powerful **Integrated Development Environment (IDE)** for Python, especially designed for:

- **Scientific computing**
- **Data science**
- **Machine learning**

It is included with **Anaconda**, so you don't need to install it separately if you have Anaconda.

Feature	Description
Editor	Where you write and edit Python code
IPython Console	Runs your code and shows results (interactive terminal)
Variable Explorer	Displays all variables in memory like Excel (great for debugging)
Plots Pane	Visualizes graphs using libraries like Matplotlib
Help Pane	Shows documentation and function details as you type

Main Features of Spyder

JUPYTER (Julia, **Py**thon, and **R)**

"A web-based interface to write and run Python code interactively."

Key Highlights of Jupyter Notebook:

- **Interactive Coding**: Run code in chunks (cells) and see results instantly.
- **Web-Based**: Opens in your browser; no complex setup required.
- **Supports Python and Other Languages**: Works primarily with Python, but also supports R, Julia, etc.
- **Rich Outputs**: Display graphs, tables, images, and more inside the notebook.
- **Mix Code and Text**: Use Markdown cells for explanations, and code cells for programs.
- **Great for Data Science & Machine Learning**: Widely used for data exploration, visualization, and prototyping models.

S. No.	DATE	NAME OF THE EXPERIMENT	PAGE No.	MARKS	SIGN
1		Decision Tree Model			
2		Support Vector Machine for E-mail Spam Detection			
3		Implementation of Facial Recognition Application			
4		Study and Implement Amazon Toolkit : Sage maker			
5		Implementation of Character Recognition			
6		Implementation of Non-Parametric Locally Weighted Regression			
7		Implementation of Sentiment Analysis Using Random Forest			
8		Demonstrate the Diagnosis of Heart Patients using Standard Heart Disease Data Set Using Bayesian Network			
9		Implementation of Online Fraud Detection using ML Algorithm			
10		Mini Project			
11		Real-Time Face Attendance System			

DECISION TREE MODEL

EX.NO: 1
DATE :

AIM:

To write a python program for the implementation of decision tree model with suitable data set and classify the data set to produce new sample.

ALGORITHM:

STEP 1: Import the necessary module for data manipulation and model selection.

STEP 2: Read the dataset(CSV file) using pandas.

STEP 3: Check for missing values in datasets, if any handle it.

STEP 4: Choose dependent(X) and independent(y) variable.

STEP 5: If there any categorical value, use encoding methods to assigns each unique value to a different integer.

STEP 6: Split the given datasets into train set and test set.

STEP 7: Choose the DecisionTreeClassifier model for making prediction.

STEP 8: Train the model to make patterns with training datasets and predict result with the test dataset.

STEP 9: Check the score of the model.

STEP 10: Use graphviz module to draw the decision tree representation in pdf format.

PROGRAM:

```
#import packages

import pandas as pd

import numpy as np

#read the data set
train = pd.read_csv("temp.csv")
```

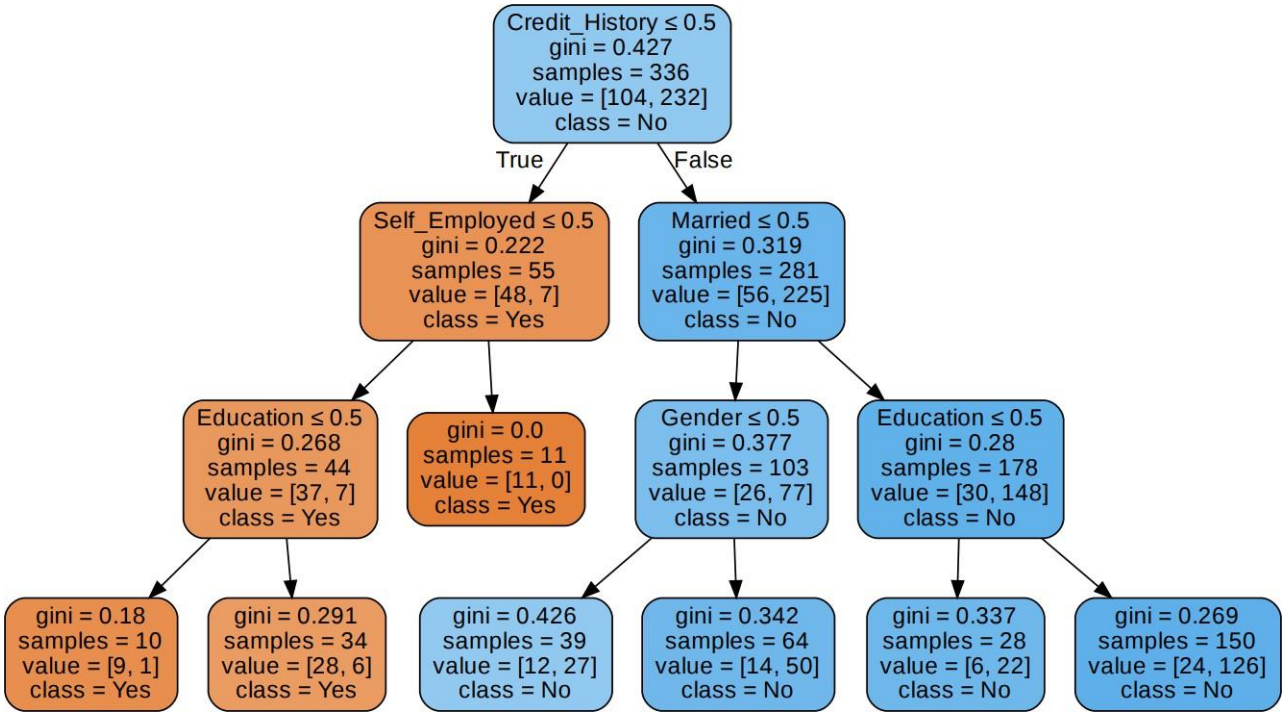
```
#check for missing values
findnull = train.isnull().sum()

#print the trained sum data
print(findnull)
#Drop null value from the dataset
train.dropna(inplace=True)
# I selected few of the columns from the dataset

train = train[['Gender','Married','Education','Self_Employed','Credit_History','Loan_Status']]

#Replace gender value replace from Male, Female to 1, 0

train['Gender']=train['Gender'].replace(to_replace='Male',value='1')
train['Gender']=train['Gender'].replace(to_replace='Female',value='0')

#Replace married status value replace from Yes, No to 1, 0

train['Married']=train['Married'].replace(to_replace='Yes',value='1')
train['Married']=train['Married'].replace(to_replace='No',value='0')

#Replace self employed status value replace from Yes, No to 1, 0

train['Self_Employed']=train['Self_Employed'].replace(to_replace='No',value='0')
train['Self_Employed']=train['Self_Employed'].replace(to_replace='Yes',value='1')

#Replace education status value replace from Graduate, Not Graduate to 1, 0

train['Education']=train['Education'].replace(to_replace='Graduate',value='1')
train['Education']=train['Education'].replace(to_replace='Not Graduate',value='0')
```

```
#Split the data-set into train and test sets
X = train.drop(columns=['Loan_Status'])
y = train.Loan_Status

from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.3,random_state=42)

#Build the model and fit the train set.
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
clf = tree.DecisionTreeClassifier(max_depth=3)
clf.fit(X_train,y_train)



#Visualize the Decision Tree
import graphviz
dot_data = tree.export_graphviz(clf, out_file=None,
                                feature_names=['Gender','Married','Education','Self_Employed','Credit_History'],
                                class_names=['Yes','No'],filled=True,
                                 rounded=True,
                                special_characters=True)
graph = graphviz.Source(dot_data)
graph.render("Gini")
graph

#Check the score of the model
output = clf.score(X_test,y_test)
print(f"The Score of the model is: {output}\nPDF")
```

OUTPUT:

The score of the model is: 0.7986111111111112

PDF:



RESULT:

Thus the above program for the implementation of decision tree model was successfully executed and verified .

SUPPORT VECTOR MACHINE FOR E-MAIL SPAM DETECTION

EX.NO: 2
DATE :

AIM:

To write a python program for the implementation of machine learning model for Email spam detection using SVM(Support Vector Machine).

ALGORITHM:

STEP 1: Import the necessary module for data manipulation and model selection.

STEP 2: Read the dataset(CSV file) using pandas.

STEP 3: Check for missing values in datasets, if any handle it.

STEP 4: Choose dependent(X) and independent(y) variable.

STEP 5: If there any categorical value, use encoding methods to assigns each unique value to a different integer.

STEP 6: Split the given datasets into train set and test set.

STEP 7: Choose the SVM model for making prediction.

STEP 8: Train the model to make patterns with training datasets and predict result with the test dataset.

STEP 9: Check the score of the model.

PROGRAM:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import CountVectorizer
from sklearn import svm

data = pd.read_csv("spam.csv")
print(data.head())
print(data.tail())
```

```
print(data.info())
#print(data.describe())
#print(data.isnull())

X = data['EmailText'].values
y = data['Label'].values

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=0)

#print(X_train)

# Converting String to Integer
cv = CountVectorizer()
X_train = cv.fit_transform(X_train)
X_test = cv.transform(X_test)

from sklearn.svm import SVC
classifier = SVC(kernel = 'rbf', random_state = 10)
classifier.fit(X_train, y_train)
print(classifier.predict(X_test))
print("Score: ",classifier.score(X_test, y_test))
```

OUTPUT:

Score: 0.9766816143497757

RESULT:

Thus the above program for the implementation of SVM model for Spam Detection was successfully executed and verified.

IMPLEMENTATION OF FACIAL RECOGNITION APPLICATION

EX.NO: 3
DATE :

AIM:

To write a python program for the implementation of facial recognition application using machine learning model.

ALGORITHM:

STEP 1: Import the following libraries for facial recognition:

i) face_recognition,
ii) Imutils,
iii) Pickle,
iv) cv2,
v) os

STEP 2: Train the model by face image dataset.

STEP 3: By using CascadeClassifier model, the face recognition is done.

STEP 4: Display the result.

PROGRAM:

i) face_train.py

```
from imutils import paths

import face_recognition

import pickle

import cv2

import os

#get paths of each file in folder named Images

#Images here contains my data(folders of various persons)

imagePaths = list(paths.list_images('Images'))

knownEncodings = []

knownNames = []
```

```
# loop over the image paths
for (i, imagePath) in enumerate(imagePaths):
    # extract the person name from the image path
    name = imagePath.split(os.path.sep)[-2]
    # load the input image and convert it from BGR (OpenCV ordering)
    # to dlib ordering (RGB)
    image = cv2.imread(imagePath)
    rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    #Use Face_recognition to locate faces
    boxes = face_recognition.face_locations(rgb,model='hog')
    # compute the facial embedding for the face
    encodings = face_recognition.face_encodings(rgb, boxes)
    # loop over the encodings
    for encoding in encodings:
        knownEncodings.append(encoding)
        knownNames.append(name)
#save emcodings along with their names in dictionary data
data = {"encodings": knownEncodings, "names": knownNames}
#use pickle to save data into a file for later use
f = open("face_enc", "wb")
f.write(pickle.dumps(data))
f.close()
```

ii) finding_face.py:

```
import face_recognition
```

```
import imutils
import pickle
import time
import cv2
import os

#find path of xml file containing haarcascade file
cascPathface = "data/haarcascade_frontalface_alt2.xml"
# load the harcaascade in the cascade classifier
faceCascade = cv2.CascadeClassifier(cascPathface)
# load the known faces and embeddings saved in last file
data = pickle.loads(open('face_enc', "rb").read())
#Find path to the image you want to detect face and pass it here
image = cv2.imread("input/test.jpg")
rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
#convert image to Greyscale for haarcascade
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
faces = faceCascade.detectMultiScale(gray,
                                     scaleFactor=1.1,
                                     minNeighbors=5,
                                     minSize=(60, 60),
                                     flags=cv2.CASCADE_SCALE_IMAGE)

# the facial embeddings for face in input
encodings = face_recognition.face_encodings(rgb)
names = []
# loop over the facial embeddings incase
# we have multiple embeddings for multiple fcaes
for encoding in encodings:
    #Compare encodings with encodings in data["encodings"]
    #Matches contain array with boolean values and True for the embeddings it matches closely
    #and False for rest
    matches = face_recognition.compare_faces(data["encodings"],
    encoding)
```

```
#set name =inknown if no encoding matches
name = "Unknown"
# check to see if we have found a match
if True in matches:
    #Find positions at which we get True and store them
    matchedIdxs = [i for (i, b) in enumerate(matches) if b]
    counts = {}
    # loop over the matched indexes and maintain a count for
    # each recognized face face
    for i in matchedIdxs:
        #Check the names at respective indexes we stored in matchedIdxs
        name = data["names"][i]
        #increase count for the name we got
        counts[name] = counts.get(name, 0) + 1
        #set name which has highest count
        name = max(counts, key=counts.get)

    # update the list of names
    names.append(name)
    # loop over the recognized faces
    for ((x, y, w, h), name) in zip(faces, names):
        # rescale the face coordinates
        # draw the predicted face name on the image
        cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 0), 2)
        cv2.putText(image, name, (x, y), cv2.FONT_HERSHEY_SIMPLEX,
         0.75, (0, 255, 0), 2)
cv2.imshow("Frame", image)
cv2.waitKey(0)
```

OUTPUT:

RESULT:

Thus the above program for the implementation of facial recognition application using machine learning model was successfully executed and verified.

STUDY AND IMPLEMENT AMAZON TOOLKIT: SAGEMAKER

EX.NO: 4
DATE :

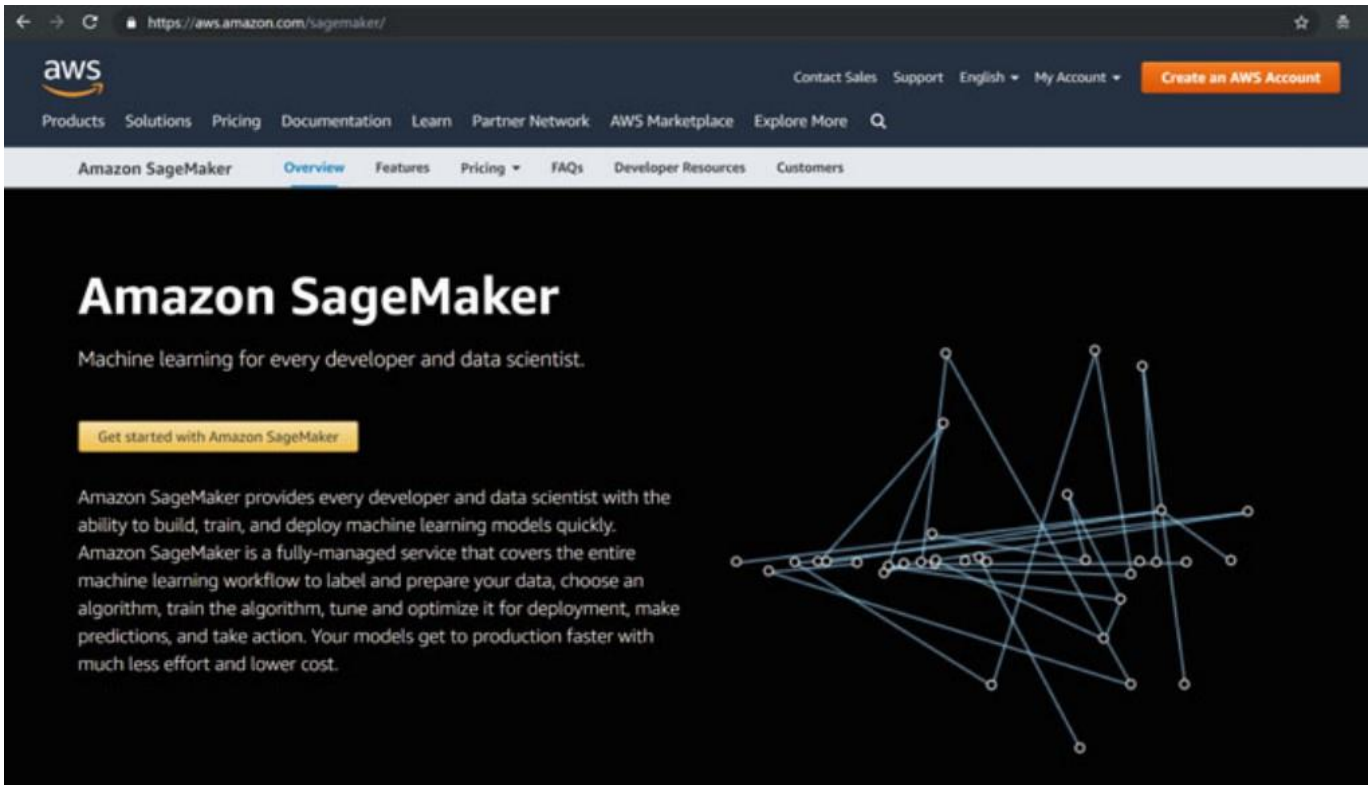
AIM:

To understand the implementation of machine learning model in amazon toolkit SEGEMAKER.

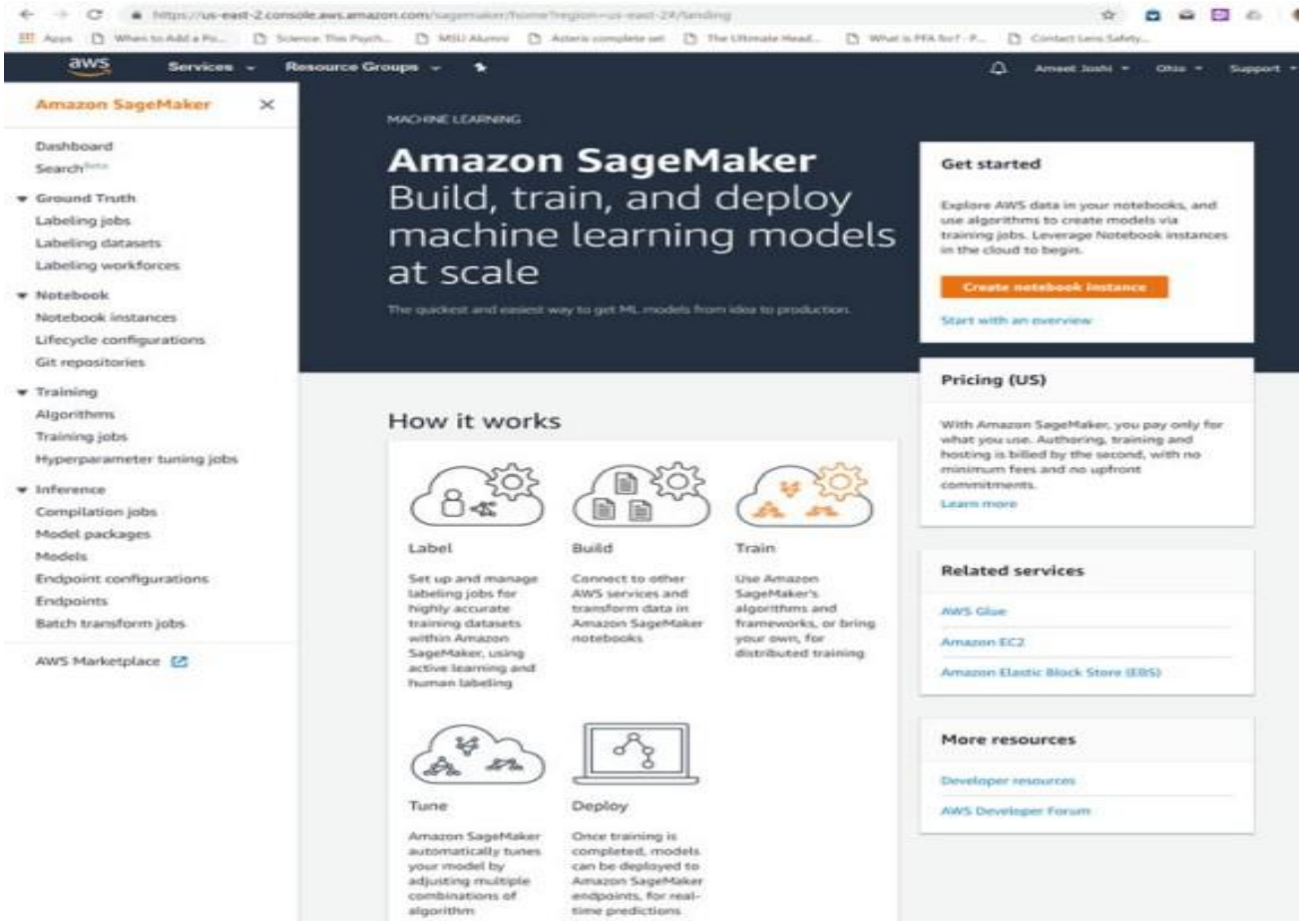
PROCEDURE:

Sagemaker is Amazon's version of a free to use framework for implementing and productizing machine learning models. It offers Amazon's own implementation of a library of models along with open source models.

Setting Up Sagemaker:

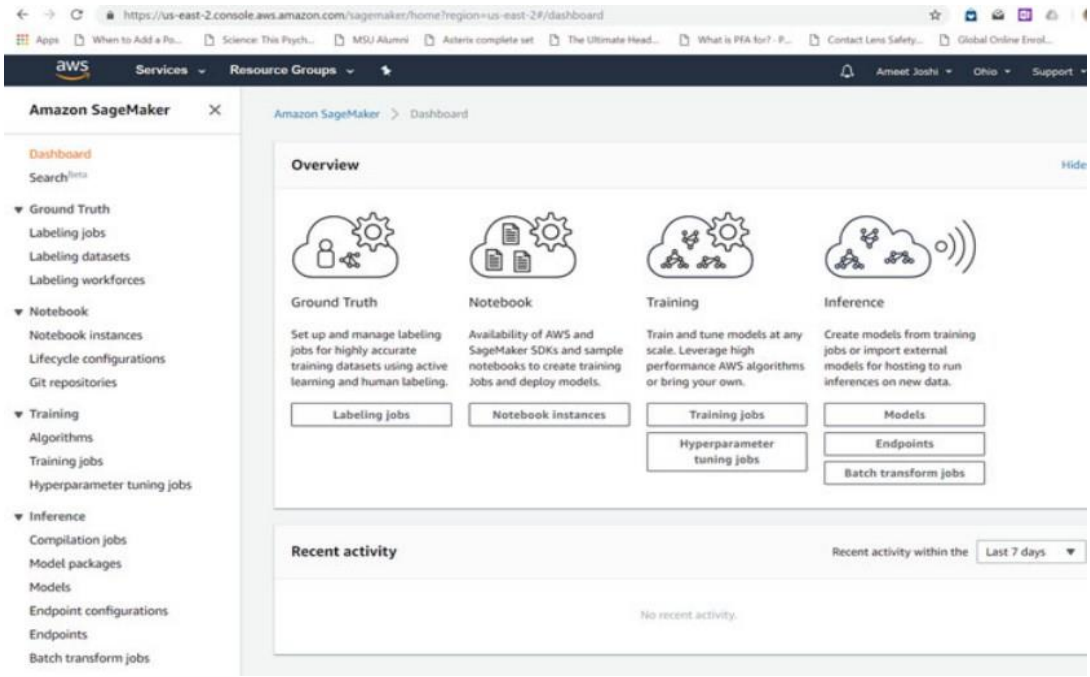


- Once you create your free account, you can start with building your first machine learning pipeline. Sagemaker interface is a little bit more advanced with a stronger focus on productizing the machine learning pipeline.



- The dashboard shows the various options you can do inside the Sagemaker.

- It also shows the previous activity. Currently it is empty, as we are just getting started. However, in the future, it will show the status of the model execution here.



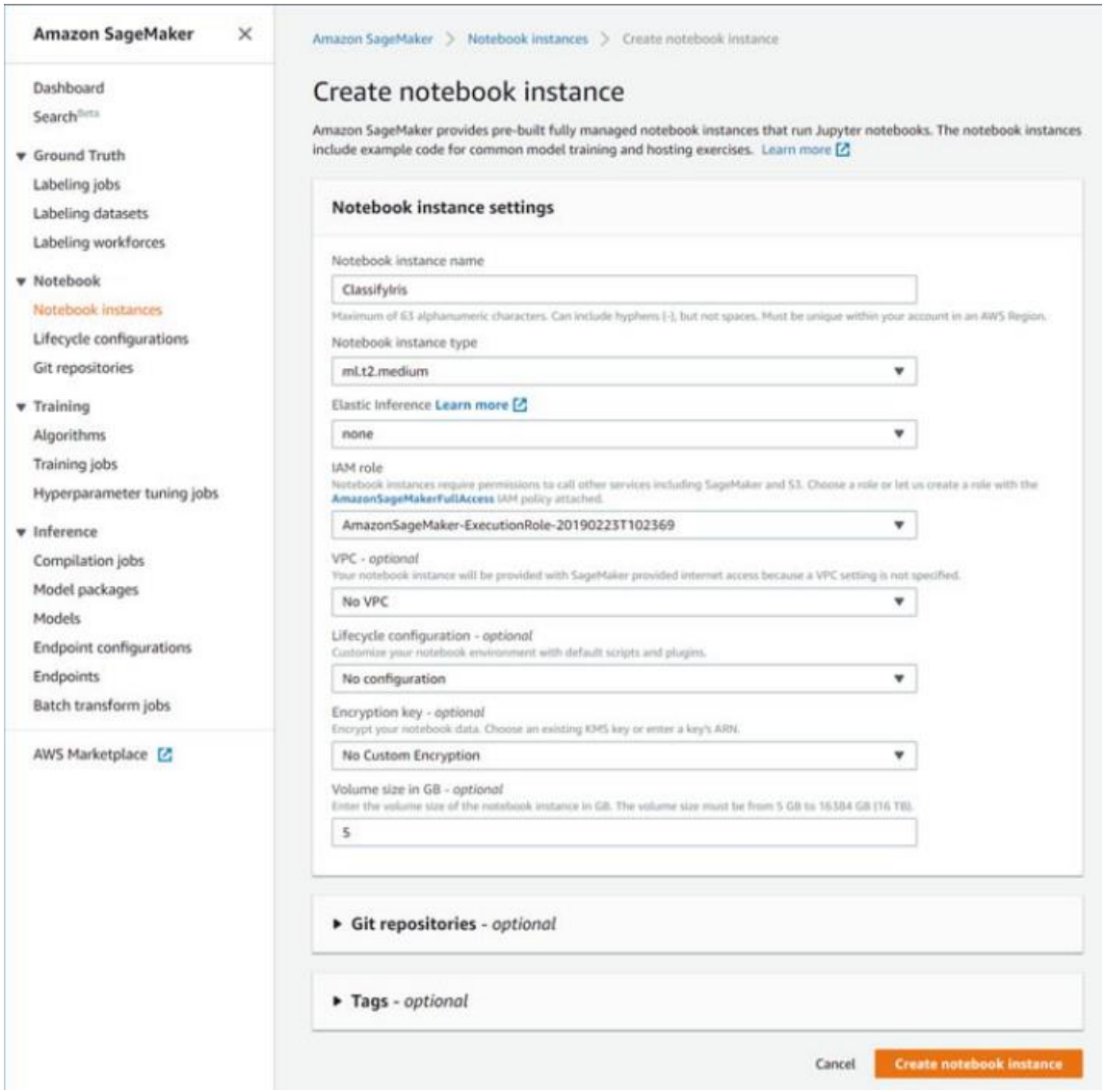
- Then a customization screen is shown to create the notebook instance.

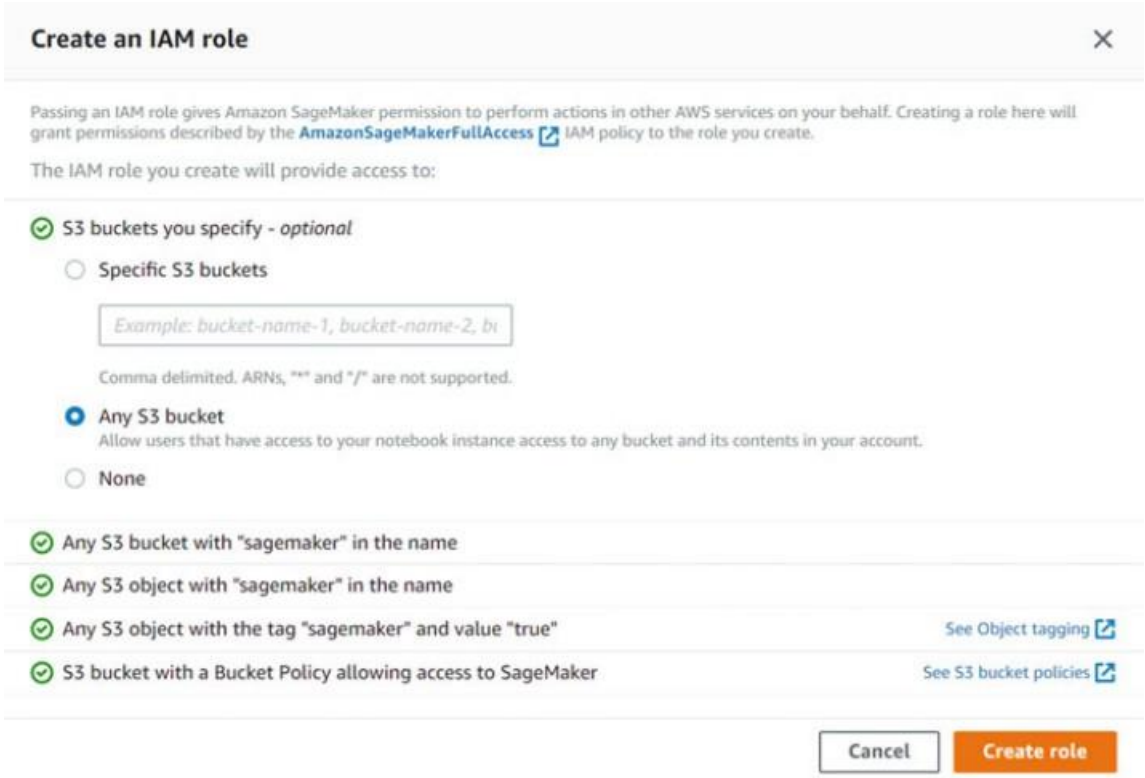
- We will name the notebook as ClassifyIris. It should be noted that underscore character is not allowed in this name. Most of the other options can be kept at default.

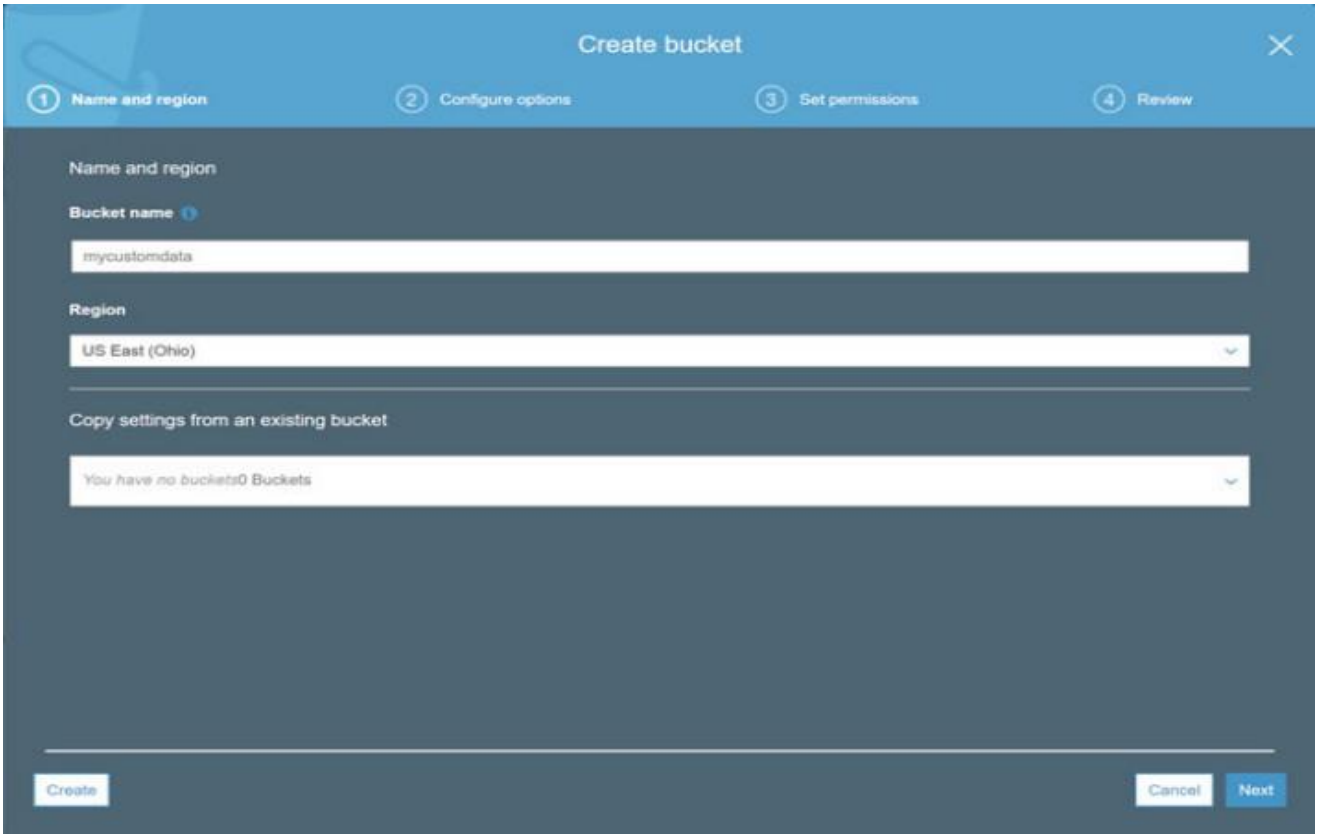
- It should be noted that 5 GB is minimum required volume, even if we are going to need only a small fraction of it. The IAM role needs to be created. This role enforces access restrictions.

- These would be useful in productizing the model later on. For now, we will just create a simple role. We will select the role that can access any bucket in the linked S3 storage.

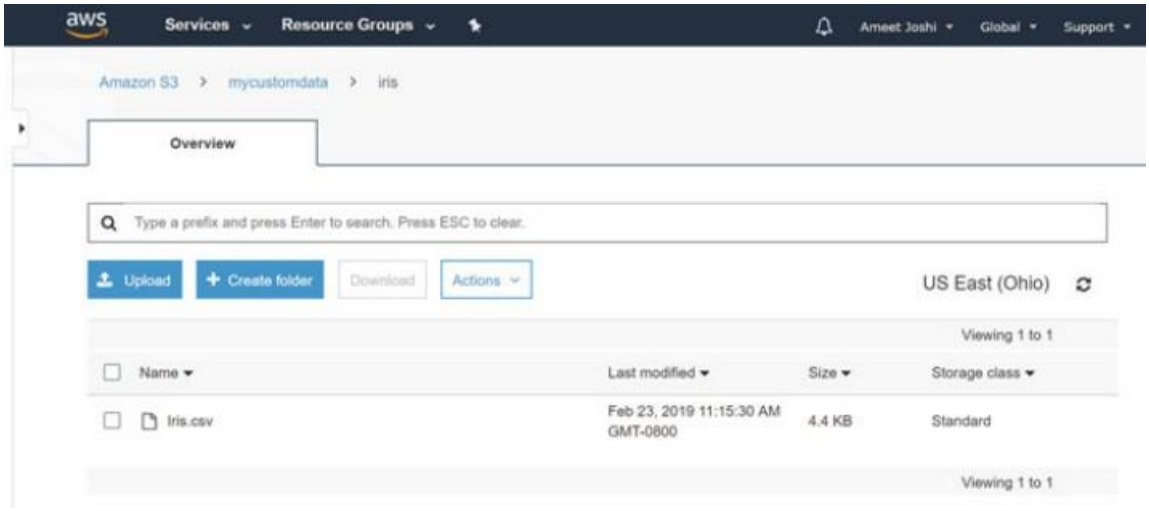
- Then we can go ahead with notebook creation. Once clicked on Create Notebook Instance, you are shown with the confirmation. The status is pending, but within a few minutes, it should be complete.







- The next step is to onboard the data. For that we need to step out of Sagemaker interface and go to list of services offered by AWS, and select S3 storage. You first need to add a bucket as the primary container of the data. We will name the bucket as mycustomdata. We will accept the default options in the subsequent screens to complete the creation of the bucket. Then we will create a folder inside the bucket with name iris.



RESULT:

Thus we have implemented the amazon toolkit: Sagemaker successfully.

IMPLEMENTATION OF CHARACTER RECOGNITION

Ex.NO: 5

DATE:

AIM:

To write a python program for the implementation of character recognition using Multilayer perceptron.

ALGORITHM:

STEP 1: Import the necessary module for data manipulation and model selection.

STEP 2: Declare a dictionary whose key represent character and values represent its assigned values.

STEP 3: Import the testing character (letter A-Z) image using open CV.

STEP 4: Predict the character and display the result.

PROGRAM:

```
import cv2
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np

from keras.models import load_model
model = load_model('handwritten_character_recog_model.h5')

words =
{0:'A',1:'B',2:'C',3:'D',4:'E',5:'F',6:'G',7:'H',8:'I',9:'J',10:'K',11:'L',12:'M',13:'N',14:'O',15:'P',16:'Q',17:'R',18:'S'
,19:'T',20:'U',21:'V',22:'W',23:'X', 24:'Y',25:'Z'}

image = cv2.imread('L.png')
image_copy = image.copy()
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
image = cv2.resize(image, (400,440))

image_copy = cv2.GaussianBlur(image_copy, (7,7), 0)
gray_image = cv2.cvtColor(image_copy, cv2.COLOR_BGR2GRAY)
_, img_thresh = cv2.threshold(gray_image, 100, 255, cv2.THRESH_BINARY_INV)
```

```
final_image = cv2.resize(img_thresh, (28,28))
final_image =np.reshape(final_image, (1,28,28,1))


prediction = words[np.argmax(model.predict(final_image))]


cv2.putText(image, "Prediction: " + prediction, (20,410), cv2.FONT_HERSHEY_DUPLEX, 1.3, color =
(0,255,0))
cv2.imshow('Project Gurukul handwritten character recognition ', image)


while (1):
    k = cv2.waitKey(1) & 0xFF
    if k == 27:
        break
cv2.destroyAllWindows()
```

OUTPUT:

RESULT:

Thus the above program for the implementation of character recognition using multilayer perceptron was successfully executed and verified.

IMPLEMENTATION OF NON-PARAMETRIC LOCALLY WEIGHTED REGRESSION

EX.NO: 6
DATE :

AIM:

To write a python program for the implementation of non-parametric Locally Weighted Regression algorithm in order to fit data points

ALGORITHM:

STEP 1: Import the necessary module for data manipulation and model selection.

STEP 2: Read the dataset(CSV file) using pandas.

STEP 3: Check for missing values in datasets, if any handle it.

STEP 4: Choose dependent(X) and independent(y) variable.

STEP 5: If there any categorical value, use encoding methods to assigns each unique value to a different integer.

STEP 6: Write function to calculate W weight diagonal Matrix used in calculation of predictions.

STEP 7: Write function to make predictions.

STEP 8: Write visualise predicted values with respect to original target values.

STEP 9: Plot the result.

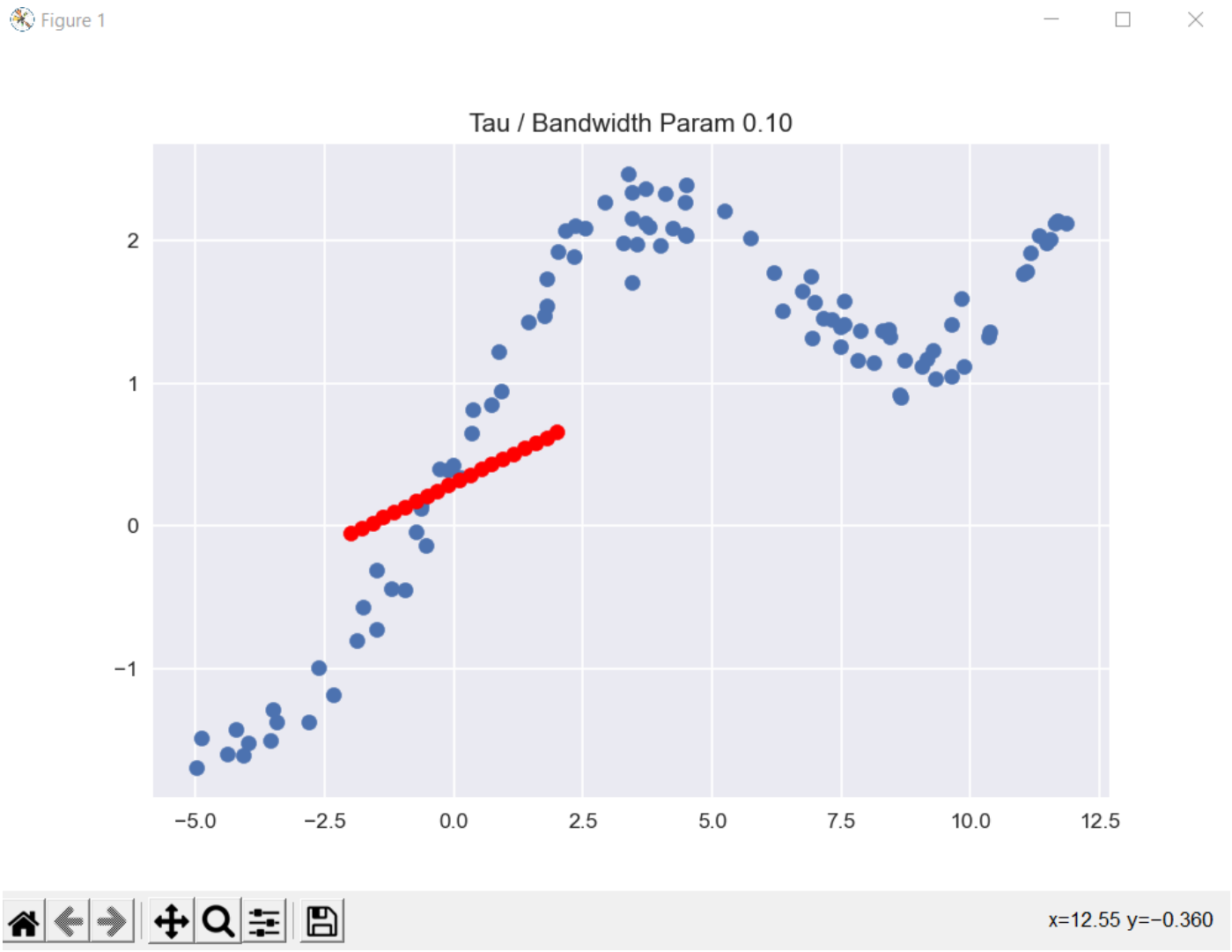
PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
plt.style.use("seaborn")
# Loading CSV files from local storage
dfx = pd.read_csv('weightedX.csv')
dfy = pd.read_csv('weightedY.csv')
# Getting data from DataFrame Object and storing in numpy n-dim arrays
X = dfx.values
```

```
Y = dfy.values


# function to calculate W weight diagonal Matrix used in calculation of predictions
def get_WeightMatrix_for_LOWES(query_point, Training_examples, Bandwidth):
    # M is the No of training examples
    M = Training_examples.shape[0]
    # Initialising W with identity matrix
    W = np.mat(np.eye(M))
    # calculating weights for query points
    for i in range(M):
        xi = Training_examples[i]
        denominator = (-2 * Bandwidth * Bandwidth)
        W[i, i] = np.exp(np.dot((xi-query_point), (xi-query_point).T)/denominator)
        return W
# function to make predictions
def predict(training_examples, Y, query_x, Bandwidth):
    M = training_examples.shape[0]
    all_ones = np.ones((M, 1))
    X_ = np.hstack((training_examples, all_ones))
    qx = np.mat([query_x, 1])
    W = get_WeightMatrix_for_LOWES(qx, X_, Bandwidth)
    # calculating parameter theta
    theta = np.linalg.pinv(X_.T*(W * X_))*(X_.T*(W * Y))
    # calculating predictions
    pred = np.dot(qx, theta)
    return theta, pred
# visualise predicted values with respect
# to original target values


Bandwidth = 0.1
X_test = np.linspace(-2, 2, 20)
Y_test = []
for query in X_test:
    theta, pred = predict(X, Y, query, Bandwidth)
```

```
    Y_test.append(pred[0][0])
horizontal_axis = np.array(X)
vertical_axis = np.array(Y)
plt.title("Tau / Bandwidth Param %.2f"% Bandwidth)
plt.scatter(horizontal_axis, vertical_axis)
Y_test = np.array(Y_test)
plt.scatter(X_test, Y_test, color ='red')
plt.show()
```

OUTPUT:



RESULT:

Thus the above program for the implementation of non-parametric Locally Weighted Regression algorithm in order to fit data points was successfully executed and verified.

IMPLEMENTATION SENTIMENT ANALYSIS USING RANDOM FOREST

EX.NO: 7
DATE :

AIM:

To write a python program for the implementation of sentiment analysis using Random Forest optimization algorithm.

ALGORITHM:

STEP 1: Import the necessary module for data manipulation and model selection.

STEP 2: Load the CSV file using pandas.

STEP 3: By using Natural language processing (NLP) convert the text into machine understanding form.

STEP 4: Morphological Analysis ⇨ Lexical Analysis ⇨ Syntactic Analysis ⇨ Semantic Analysis is done by using NLP module.

STEP 5: Encoding - convert the given text into machine understandable form.

STEP 6: By using Random Forest model Sentiment of the twitter data is analayzed.

STEP 7: Plot the confusion matrix for the twitter data.

STEP 8: Find the score of the model and display the result.

PROGRAM:

```
import pandas as pd

train_df = pd.read_csv("train.csv")

train_df.head()

test_df = pd.read_csv("train.csv")

test_df.head()

import re

CONTRACTIONS_DICT = {"can`t":"can not",

                     "won`t":"will not",
```

```
                "don`t":"do not",
                "aren`t":"are not",
                "i`d":"i would",
                "couldn`t": "could not",
                "shouldn`t": "should not",
                "wouldn`t": "would not",
                "isn`t": "is not",
                "it`s": "it is",
                "didn`t": "did not",
                "weren`t": "were not",
                "mustn`t": "must not",
               }
def prepare_data(df:pd.DataFrame) -> pd.DataFrame:
    df["text"] = df["text"] \
            .apply(lambda x: re.split('http:\/\/.*', str(x))[0]) \
            .str.lower() \
            .apply(lambda x: replace_words(x,CONTRACTIONS_DICT))

    df["label"] = df["sentiment"].map(
                    {"neutral": 1, "negative":0, "positive":2 }
                    )
    return df.text.values, df.label.values
def replace_words(string:str, dictionary:dict):
    for k, v in dictionary.items():
        string = string.replace(k, v)
    return string
train_tweets, train_labels = prepare_data(train_df)
```

```
test_tweets, test_labels = prepare_data(test_df)
from keras.preprocessing.text import Tokenizer
tokenizer = Tokenizer()
tokenizer.fit_on_texts(train_tweets)
train_tokenized = tokenizer.texts_to_matrix(
                        train_tweets,
                        mode='tfidf'
                        )

print(train_tokenized)

test_tokenized = tokenizer.texts_to_matrix(
                        test_tweets,
                        mode='tfidf'
                        )
print(test_tokenized)
from  sklearn.ensemble import RandomForestClassifier
forest = RandomForestClassifier(
                          n_estimators=500,
                          min_samples_leaf=2,
                          oob_score=True,
                          n_jobs=-1,
                          )
forest.fit(train_tokenized,train_labels)
print(f"Train score: {forest.score(train_tokenized,train_labels)}")
print(f"OOB score: {forest.oob_score_}")
```

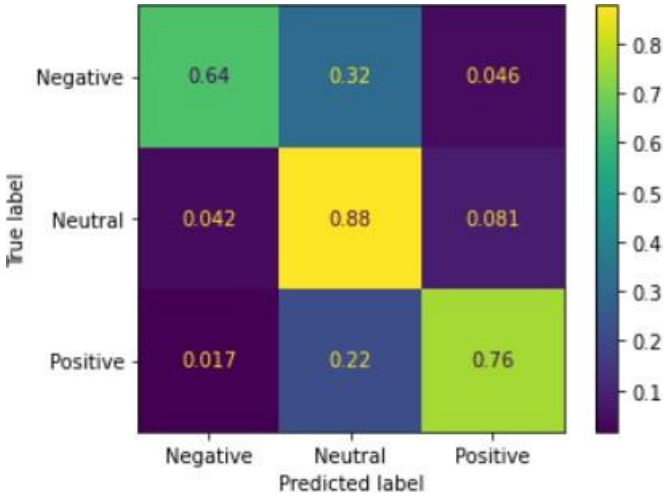
```
print(f"Test score: {forest.score(test_tokenized,test_labels)}")

from sklearn.metrics import plot_confusion_matrix

plot_confusion_matrix(
    forest,
    test_tokenized,
    test_labels,
    display_labels=["Negative","Neutral","Positive"],
    normalize='true'
)
```

OUTPUT:

Train score: 0.773977397739774

OOB score: 0.6635663566356635

Test score: 0.773977397739774



RESULT:

Thus the above program for the implementation of the Sentiment Analysis using Random Forest algorithm successfully executed and verified.

DEMONSTRATE THE DIAGNOSIS OF HEART PATIENTS USING STANDARD HEART DISEASE DATA SET UISNG BAYESIAN NETWORK

EX.NO: 8
DATE :

AIM:

To write a python program for the demonstration of diagnosis of heart patients using Bayesian network.

ALGORITHM:

STEP 1: Open VS Code.

STEP 2: Install the required packages.

STEP 3: Read, import, transform and train the heart disease dataset.

STEP 4: Construct the Bayesian network

STEP 5: Calculate the probability of occurrence of heart disease and print it

PROGRAM:

```
import numpy as np

import csv

import pandas as pd


from pgmpy.models import BayesianModel

from pgmpy.estimators import MaximumLikelihoodEstimator

from pgmpy.inference import VariableElimination


heartDisease = pd.read_csv('heart.csv')

heartDisease = heartDisease.replace('?',np.nan)

print('Few examples from the dataset are given below')
```

```
print(heartDisease.head())


Model=BayesianModel([('age','trestbps'),('age','fbs'),('sex','trestbps'),('exang','trestbps'),('trestbps','heartdise
ase'),('fbs','heartdisease'),('heartdisease','restecg'),('heartdisease','thalach'),('heartdisease','chol')])
print('\n Learning CPD using Maximum likelihood estimators')
model.fit(heartDisease,estimator=MaximumLikelihoodEstimator)
print('\n Inferencing with Bayesian Network:')
HeartDisease_infer = VariableElimination(model)


print('\n 1. Probability of HeartDisease given Age=30')
q=HeartDisease_infer.query(variables=['heartdisease'],evidence={'age':28})
print(q['heartdisease'])
print('\n 2. Probability of HeartDisease given cholesterol=100')
q=HeartDisease_infer.query(variables=['heartdisease'],evidence={'chol':100})
print(q['heartdisease'])
```

OUTPUT:

```
Few examples from the dataset are given below
age sex cp trestbps ...slope ca thal heartdisease
0 63 1 1 145 ... 3 0 6 0
1 67 1 4 160 ... 2 3 3 2
2 67 1 4 120 ... 2 2 7 1
3 37 1 3 130 ... 3 0 3 0
4 41 0 2 130 ... 1 0 3 0
[5 rows x 14 columns]
Learning CPD using Maximum likelihood estimators
Inferencing with Bayesian Network:
1. Probability of HeartDisease given Age=28
╒═══════════════════╤═══════════════════════╕
│ heartdisease      │ phi(heartdisease)     │
╞═══════════════════╪═══════════════════════╡
│ heartdisease_0    │ 0.6791                │
```

heartdisease_1	0.1212
heartdisease_2	0.0810
heartdisease_3	0.0939
heartdisease_4	0.0247

2. Probability of HeartDisease given cholesterol=100

heartdisease	phi(heartdisease)
heartdisease_0	0.5400
heartdisease_1	0.1533
heartdisease_2	0.1303
heartdisease_3	0.1259
heartdisease_4	0.0506

RESULT:

Thus the program to construct a Bayesian network for Heart Disease Data Set is executed successfully.

IMPLEMENTATION OF ONLINE FRAUD DETECTION USING ML ALGORITHM

EX.NO: 9
DATE :

AIM:

To write a python program for the implementation of online fraud detection using ML algorithm.

ALGORITHM:

STEP 1: Import the necessary module for data manipulation and model selection

STEP 2: Load the CSV file using pandas

STEP 3: Split the data into label and the value part

STEP 4: By using Plotly express ,Pie chart is drawn.

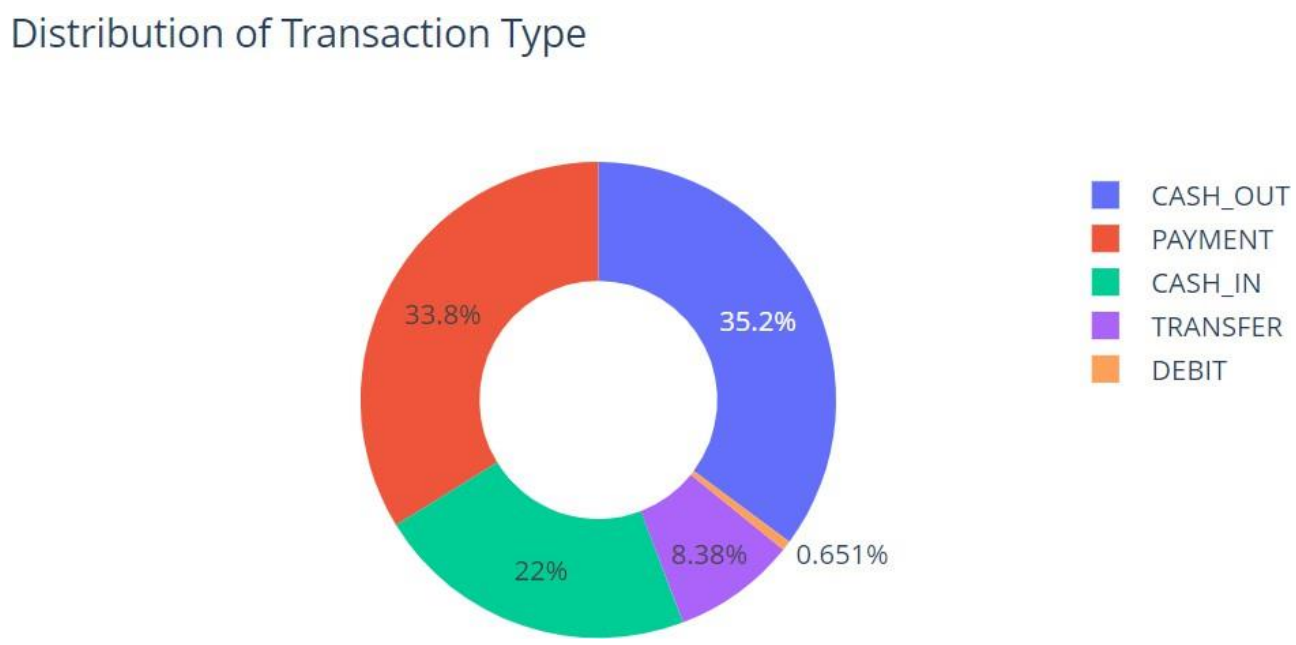
STEP 5: The analyse is made by DecisionTreeClassifier ML algorithm.

PROGRAM:

```
import pandas as pd
import numpy as np
data = pd.read_csv("dataset.csv")
print(data.head())
print(data.type.value_counts())
type = data["type"].value_counts()
transactions = type.index
quantity = type.values

import plotly.express as px
figure = px.pie(data,
        values=quantity,
        names=transactions,hole = 0.5,
        title="Distribution of Transaction Type")
figure.show()
data["type"] = data["type"].map({"CASH_OUT": 1, "PAYMENT": 2,
```

```
                    "CASH_IN": 3, "TRANSFER": 4,
                    "DEBIT": 5})
data["isFraud"] = data["isFraud"].map({0: "No Fraud", 1: "Fraud"})
print(data.head())
from sklearn.model_selection import train_test_split
x = np.array(data[["type", "amount", "oldbalanceOrg", "newbalanceOrig"]])
y = np.array(data[["isFraud"]])
from sklearn.tree import DecisionTreeClassifier
xtrain, xtest, ytrain, ytest = train_test_split(x, y, test_size=0.10, random_state=42)
model = DecisionTreeClassifier()
model.fit(xtrain, ytrain)
print(model.score(xtest, ytest))
features = np.array([[4, 9000.60, 9000.60, 0.0]])
print(model.predict(features))
```

OUTPUT:



RESULT:

Thus the above program for the implementation of online fraud detection using ML is executed successfully

MINI PROJECT (RECOMMENDATION SYSTEM)

EX.NO: 10

DATE :

AIM:

To write a python program for the implementation of Recommendation system using machine learning algorithm.

ALGORITHM:

STEP 1: Import the necessary module for data manipulation and model selection

STEP 2: Load the CSV file using pandas

STEP 3: Recommendation are made by Content-Based Recommendation System and Ranking methods

STEP 4: KNN algorithm is used to build a Recommendation system model

STEP 5: Result are displayed

PROGRAM:

```
import numpy as np
import pandas as pd
import sklearn
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
ratings = pd.read_csv("ratings.csv")
ratings.head()
movies = pd.read_csv("movies.csv")
movies.head()
n_ratings = len(ratings)
n_movies = len(ratings['movieId'].unique())
n_users = len(ratings['userId'].unique())
```

```
print(f"Number of ratings: {n_ratings}")
print(f"Number of unique movieId's: {n_movies}")
print(f"Number of unique users: {n_users}")
print(f"Average ratings per user: {round(n_ratings/n_users, 2)}")
print(f"Average ratings per movie: {round(n_ratings/n_movies, 2)}")
user_freq = ratings[['userId', 'movieId']].groupby('userId').count().reset_index()
user_freq.columns = ['userId', 'n_ratings']
user_freq.head()
mean_rating = ratings.groupby('movieId')[['rating']].mean()

lowest_rated = mean_rating['rating'].idxmin()
movies.loc[movies['movieId'] == lowest_rated]
highest_rated = mean_rating['rating'].idxmax()
movies.loc[movies['movieId'] == highest_rated]
ratings[ratings['movieId']==highest_rated]
ratings[ratings['movieId']==lowest_rated]
movie_stats = ratings.groupby('movieId')[['rating']].agg(['count', 'mean'])
movie_stats.columns = movie_stats.columns.droplevel()
from scipy.sparse import csr_matrix
def create_matrix(df):
    N = len(df['userId'].unique())
    M = len(df['movieId'].unique())
    user_mapper = dict(zip(np.unique(df["userId"]), list(range(N))))
    movie_mapper = dict(zip(np.unique(df["movieId"]), list(range(M))))
    user_inv_mapper = dict(zip(list(range(N)), np.unique(df["userId"])))
    movie_inv_mapper = dict(zip(list(range(M)), np.unique(df["movieId"])))
```

```
    user_index = [user_mapper[i] for i in df['userId']]
    movie_index = [movie_mapper[i] for i in df['movieId']]
    X=csr_matrix((df["rating"], (movie_index, user_index)), shape=(M, N))
    return X, user_mapper, movie_mapper, user_inv_mapper, movie_inv_mapper

X, user_mapper, movie_mapper, user_inv_mapper, movie_inv_mapper = create_matrix(ratings)

from sklearn.neighbors import NearestNeighbors
def find_similar_movies(movie_id, X, k, metric='cosine', show_distance=False):
    neighbour_ids = []
    movie_ind = movie_mapper[movie_id]
    movie_vec = X[movie_ind]
    k+=1
    kNN = NearestNeighbors(n_neighbors=k, algorithm="brute", metric=metric)
    kNN.fit(X)
    movie_vec = movie_vec.reshape(1,-1)
    neighbour = kNN.kneighbors(movie_vec, return_distance=show_distance)
    for i in range(0,k):
    n = neighbour.item(i)
    neighbour_ids.append(movie_inv_mapper[n])
    neighbour_ids.pop(0)
    return neighbour_ids

movie_titles = dict(zip(movies['movieId'], movies['title']))
movie_id = 3
similar_ids = find_similar_movies(movie_id, X, k=10)
movie_title = movie_titles[movie_id]

print(f"Since you watched {movie_title}")
for i in similar_ids:
    print(movie_titles[i])
```

OUTPUT:

Number of ratings: 100836

Number of unique movieId's: 9724

Number of unique users: 610

Average ratings per user: 165.3

Average ratings per movie: 10.37

Since you watched Grumpier Old Men (1995)

Grumpy Old Men (1993)

Striptease (1996)

Nutty Professor, The (1996)

Twister (1996)

Father of the Bride Part II (1995)

Broken Arrow (1996)

Bio-Dome (1996)

Truth About Cats & Dogs, The (1996)

Sabrina (1995)

Birdcage, The (1996)

RESULT:

Thus the above program for the implementation of Recommendation system using ML was executed successfully and verified.

EX.NO: 11 **REAL-TIME FACE ATTENDANCE SYSTEM**

DATE :

AIM:

Detect a face and capture the data then store it as a pkl file for attendance system.

ALGORITHM:

1.Initialize System: Start video capture and load Haar Cascade for face detection.

2.Collect Face Data: Capture user's face images, resize to (50x50),and store until 100 images are collected.

3.Save Data: Convert face data to a NumPy array and store it in pickle files (face_data.pkl & names.pkl).

4.Load Data & Train Model: Retrieve stored face data and labels, then train a K-Nearest Neighbors (KNN) classifier.

5.Face Recognition: Continuously capture frames, detect faces, and classify them using the trained KNN model.

6.Display Results: Draw rectangles around detected faces and display the predicted name on the screen.

7.Record Attendance: On pressing 'o', save the recognized name along with the timestamp and date n a CSV file.

8.Exit System: On pressing 'q', stop video capture, release resources, and close all windows

PROGRAM

```
import cv2

videos = cv2.VideoCapture(0) if not videos.isOpened():

print("Error: Could not open video device.") exit()

facedetect = cv2.CascadeClassifier('haarcascade_frontalface_default.xml') if facedetect.empty():

print("Error: Could not load Haar cascade file.")

exit()

name = input("Enter your name: ").strip() if not name:

print("Error: Name cannot be empty.")

exit() import os

os.makedirs('data', exist_ok=True)
```

```
import numpy as np
face_data = [] i = 0
print("Press 'q' to quit or wait until 100 face captures are complete.") while True:
ret, frame = videos.read()
if not ret:
print("Failed to capture frame. Exiting...") break
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY) faces = facedetect.detectMultiScale(gray, 1.3, 5)
for (x, y, w, h) in faces:
crop_img = frame[y:y+h, x:x+w] try:
resized_img = cv2.resize(crop_img, (50, 50))
except Exception as e:
print("Error resizing face image:", e) continue
if len(face_data) < 100 and i % 10 == 0: face_data.append(resized_img)
cv2.putText(frame, f"Captured: {len(face_data)}", (50, 50), cv2.FONT_HERSHEY_COMPLEX, 1, (255,
0, 0), 1)
cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
cv2.imshow('Face Capture', frame) i += 1
if len(face_data) >= 100: print("Capture Complete!")
cv2.putText(frame, "Capture Complete!", (50, 100), cv2.FONT_HERSHEY_COMPLEX, 1, (0, 255,
0), 2)
cv2.imshow('Face Capture', frame) cv2.waitKey(2000) # Show the message for 2 seconds break
k = cv2.waitKey(1) if k == ord('q'):
print("Exiting capture early.")
break
videos.release() cv2.destroyAllWindows()
import pickle
# Convert face data to numpy array face_data = np.array(face_data)
face_data = face_data.reshape(len(face_data), -1)
# Load or initialize names list
if os.path.exists('data/names.pkl'): with open('data/names.pkl', 'rb') as f: names = pickle.load(f)
```

```
else:
names = []
names.extend([name] * len(face_data)) # Save updated names
with open('data/names.pkl', 'wb') as f:
pickle.dump(names, f)
# Load or initialize face data
if os.path.exists('data/face_data.pkl'): with open('data/face_data.pkl', 'rb') as f:
faces = pickle.load(f)
faces = np.append(faces, face_data, axis=0) else:
faces = face_data
# Save updated face data
with open('data/face_data.pkl', 'wb') as f: pickle.dump(faces, f)
                                print("Data saved successfully.")
import cv2
import numpy as np import os
import pickle import time import csv
from sklearn.neighbors import KNeighborsClassifier from datetime import datetime
# Initialize video capture video = cv2.VideoCapture(0) if not video.isOpened():
print("Error: Could not open video device.") exit()
# Load Haar Cascade for face detection
facedetect = cv2.CascadeClassifier('haarcascade_frontalface_default.xml') if facedetect.empty():
print("Error: Could not load Haar cascade file.") exit()
# Load names and face data from pickle files
try:
with open('data/names.pkl', 'rb') as w: LABELS = pickle.load(w)
with open('data/face_data.pkl', 'rb') as f: FACES = pickle.load(f)
except FileNotFoundError:
print("Error: Required data files not found.") exit()
# Train K-Nearest Neighbors classifier
knn = KNeighborsClassifier(n_neighbors=5)
```

```
knn.fit(FACES, LABELS)
# CSV column names
COL_NAMES = ['Name', 'Time', 'Date']
print("Press 'q' to quit or 'o' to record attendance.") while True:
ret, img = video.read() if not ret:
print("Error: Failed to capture video frame.")
break
faces = facedetect.detectMultiScale(gray, 1.3, 5)
for (x, y, w, h) in faces:
# Extract the detected face region crop_img = img[y:y+h, x:x+w, :] try:
resized_img = cv2.resize(crop_img, (50, 50)).flatten().reshape(1, -1) except Exception as e:
print("Error resizing face image:", e) continue
# Predict the name of the detected face output = knn.predict(resized_img)
# Get current timestamp and date ts = time.time()
date = datetime.fromtimestamp(ts).strftime('%d-%m-%Y')
timestamp = datetime.fromtimestamp(ts).strftime('%H:%M:%S')
# Attendance record entry
attendance = [str(output[0]), str(timestamp), str(date)]
# Display rectangle and label around the detected face cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)
cv2.rectangle(img, (x, y-40), (x+w, y), (0, 255, 0), -1)
cv2.putText(img, str(output[0]), (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)
# Save attendance on pressing 'o'
if cv2.waitKey(1) & 0xFF == ord('o'):
time.sleep(1) # Small delay to prevent rapid key triggers filename = f'Attendance/attendance_{date}.csv'
file_exists = os.path.isfile(filename)
# Write attendance record
with open(filename, 'a' if file_exists else 'w', newline='') as csvFile: writer = csv.writer(csvFile)
if not file_exists: writer.writerow(COL_NAMES)
writer.writerow(attendance)
print(f"Attendance recorded for {output[0]} at {timestamp} on {date}.")
```

Show the video frame

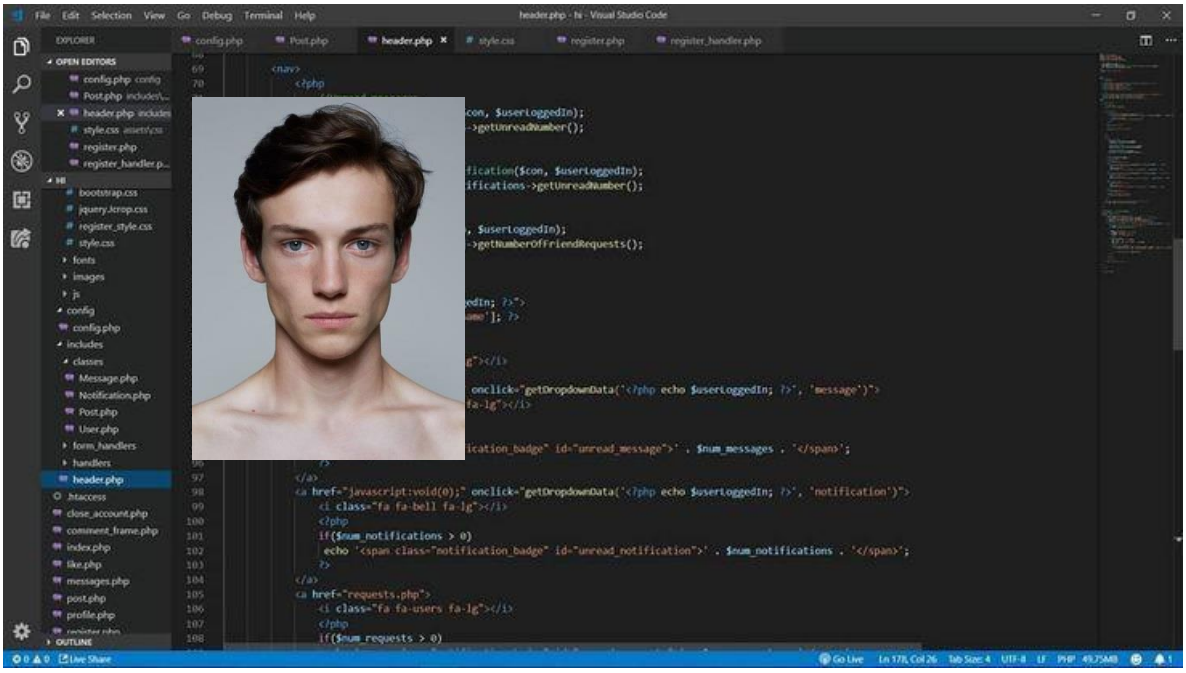
cv2.imshow('Attendance System by Nethaji', img)

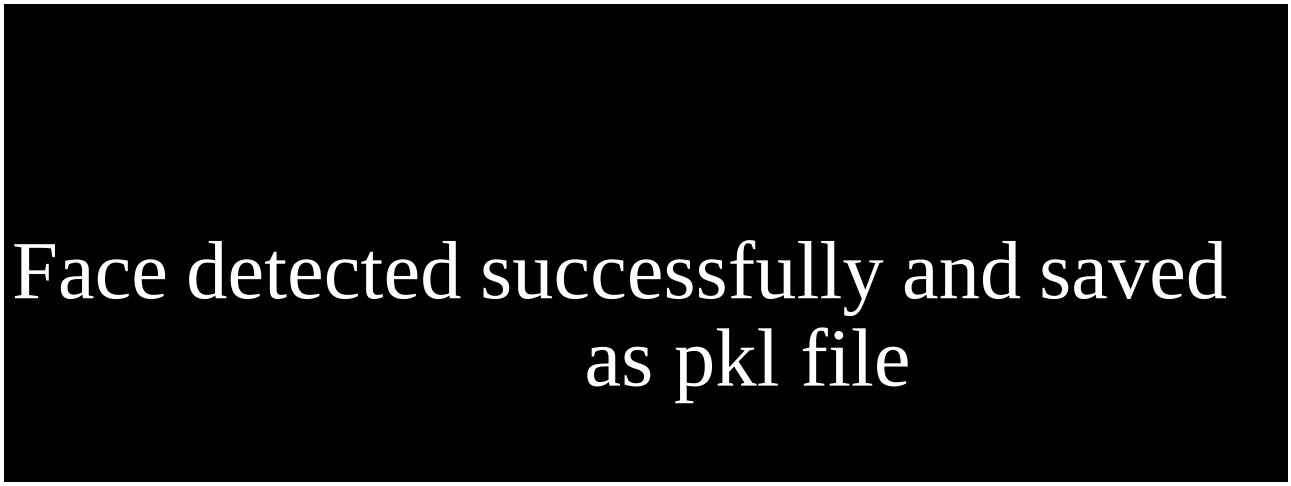
Quit the application on pressing 'q' if cv2.waitKey(1) & 0xFF == ord('q'):

break

Release resources video.release()

OUTPUT





RESULT:

Thus the program captures faces from the webcam, resizes them, and stores the face data and user name in face_data.pkl and names.pkl files for later use in face recognition